

Course-End Project

NutriWise AI: LLM-Powered Calorie Tracker

Project Statement:

NutriWise, a health-technology startup, wants to simplify nutrition tracking by allowing users to describe meals in natural language instead of manually searching food databases and entering portion sizes.

As an AI Engineer, you have been asked to build a Calorie Tracker, a lightweight application that uses Large Language Models (LLMs) to estimate calories and macronutrients from meal descriptions. The solution should provide realistic nutrition estimates, support common home-cooked meals, and clearly communicate uncertainty.

The application should use a multi-stage estimation pipeline — a local food table, free nutrition APIs, and LLM fallbacks — to generate structured nutrition estimates, maintain a daily meal log, and provide a simple user experience through a Gradio interface.

You are required to:

1. Convert free-text meal descriptions into structured nutrition estimates
2. Improve estimate reliability using a multi-stage LLM workflow
3. Produce consistent, normalized structured outputs for every estimate
4. Allow users to save meals and track daily calorie consumption
5. Store meal records locally without requiring a database
6. Handle unavailable APIs and LLM providers gracefully
7. Develop a Gradio-based user interface for estimation and tracking
8. Separate application logic and the user interface into modular components
9. Package and test the application for local deployment

Instructions:

- **Tools:** Use Jupyter, Python 3.10+, Gradio, the google-genai SDK, requests, python-dotenv, and optional nutrition or LLM API keys (Google Gemini, Groq, OpenRouter, USDA FoodData Central, or API Ninjas)
- **Dataset:** Download and use the Healthy Meal Plan Dataset from Kaggle or create your own meal inputs for testing. The application should support common home-cooked meals and natural-language food descriptions.

- **Submission:** Submit a shared Google Drive link containing the complete source code, project report, screenshots, and setup instructions

Steps to Perform:

1. Set up the environment

- Create a virtual environment and install all dependencies listed in requirements.txt
- Configure at least one provider, Google Gemini (GEMINI_API_KEY / GOOGLE_API_KEY) or OpenAI (OPENAI_API_KEY)
- Verify that the application components load successfully

2. Build the LLM agent pipeline

- Create a local food table and a parser that turns free-text meals into food items with portions
- Create the main estimation function and the local food table stage for common home-cooked and Indian meals
- Create a free nutrition API stage that looks up foods using API Ninjas, USDA FoodData Central, or Open Food Facts
- Create an LLM fallback stage that queries Gemini, then Groq, then OpenRouter when earlier stages do not resolve the meal
- Create a no-stop fallback that returns a rough placeholder estimate, so the application never crashes
- Normalize every result into a consistent structure with name, calories, protein, carbs, fat, a note, and the source used

3. Build the Gradio web interface

- Import Gradio and the estimation and storage functions, and define the column headers for the meal log
- Create display helpers that build today's saved meals as table rows and a one-line daily summary
- Create the button handlers that run an estimate, save it to the log, and remove a meal by its id
- Build the page layout with a title, caption, daily summary line, meal input box, Estimate and Save buttons, a JSON panel for the latest estimate, a saved-meals table, and a delete field, and connect each button to its handler

- Create a launch function that builds and starts the Gradio app so it can run as a web app, in a notebook, or from the command line

4. Implement data persistence and reporting

- Save each meal to data/meals.json with a UUID, ISO date, created_at timestamp, and all estimate fields, inserting every new meal at the top
- Delete a meal by its id and refresh the log immediately
- Filter records by today's date and sum calories to compute and display the daily total

5. Test and validate the application

- Verify nutrition estimates across different meal descriptions
- Test the multi-stage fallback behaviour and validate data persistence and daily tracking
- Package the application for local execution

Deliverables:

The final deliverables will include:

1. Project submission package

- A shared Google Drive link containing the complete project source code
- A README file with setup and execution instructions
- Screenshots demonstrating the application workflow and generated outputs

2. Completed application consisting of:

- LLM-powered nutrition estimation workflow
- Structured output generation using Pydantic
- Local meal logging and calorie tracking functionality
- Gradio-based nutrition assistant interface

3. Project report consisting of:

- Introduction: Project objective, problem statement, and business relevance
- Implementation Overview: LLM workflow, structured output design, and application architecture

- Results: Nutrition estimates generated, meal tracking outputs, and application walkthrough
- Conclusion: Key insights, limitations, and future enhancements